

Proyecto final

Sistema de apoyo para el mantenimiento de flotillas vehiculares

Fundamentos de programación



Por:
Luis Eduardo Reynoso Rodríguez

25 de Septiembre de 2026

Introducción

Se propone un sistema de mantenimiento para flotillas vehiculares para la empresa CEMEX, el área seleccionada para la aplicación de este sistema es el área de administración y mantenimiento de la flotilla vehicular esto debido a que es de alta importancia mantener en buen estado las unidades para poder llevar una correcta continuidad para las actividades de distribución, atención y movilidad.

El proyecto comenzó como un prototipo enfocado en consultar kilometrajes y costos de mantenimiento. En esta fase final de entrega se realiza una ampliación para incorporar identificación de usuario, pantalla de carga, un menú principal, control de inactividad, captura estructurada de fecha, persistencia en archivos de texto, validaciones de entradas y el manejo de excepciones.

La finalidad del programa es apoyar el registro y consulta de información de las unidades, identificar si un vehículo requiere mantenimiento, estimar el costo del servicio y conservar un historial básico de las operaciones realizadas.

Definición del problema

El mantenimiento de una flotilla vehicular requiere dar seguimiento a las unidades para identificar cuando es necesario el servicio o mantenimiento de una unidad.

El sistema desarrollado busca resolver esta necesidad mediante una interfaz de consola que compara el kilometraje actual con el kilometraje del próximo servicio, identifica si la unidad requiere mantenimiento y permite consultar o registrar información en archivos externos.

Reglas del programa y de la empresa

1. El usuario debe identificarse con un nombre o nickname antes de utilizar el sistema.

2. El acceso requiere la contraseña establecida en el programa.
3. Solo se contemplan las tres unidades registradas: Mazda 3, Toyota Hilux y Chevrolet Suburban.
4. La selección del vehículo acepta número de opción o nombre; además, se normalizan mayúsculas, minúsculas y espacios adicionales considerando cualquier tipo de diferencias al momento en el que el usuario ingresa la unidad a trabajar.
5. Los kilometrajes deben ser números enteros positivos y mayores que cero.
6. Si el kilometraje actual es mayor o igual al kilometraje del próximo servicio, la unidad requiere mantenimiento.
7. Si todavía no se alcanza el kilometraje del próximo servicio, el sistema calcula los kilómetros faltantes.
8. Cuando se requiere mantenimiento, el usuario puede seleccionar mantenimiento menor o mantenimiento mayor.
9. Una opción de mantenimiento inválida se rechaza antes de calcular o asignar un costo.
10. La fecha se almacena en una tupla con la sintaxis Fecha = día, mes, año y se integra en los registros escritos en archivos.
11. La fecha se valida considerando los días reales de cada mes y los años bisiestos.
12. El sistema trabaja con cuatro archivos de texto de prueba y permite consultar y anexar información de forma permanente.
13. Los errores de archivos se controlan mediante try-except para evitar cierres abruptos.

Funcionamiento requerido del código

1. Identificación de usuario:

Al iniciar, el sistema solicita el nombre o nickname mediante `entrada_usuario()`. También se evita que el campo quede vacío.

2. Bienvenida dinámica:

Después de haberse validado el nombre se genera el mensaje de bienvenida incorporando el texto ingresado por el usuario.

3. Pantalla de carga:

La función carga() imprime un aviso y utiliza time.sleep(1) dentro de un for de tres repeticiones. La duración total es de aproximadamente tres segundos, por debajo del máximo de cinco.

4. Menú como matriz con while:

Las opciones se almacenan en la variable menu como una matriz de pares número-descripción. El menú se presenta dentro del ciclo while menuact == True.

5. Control de inactividad:

La función entrada_usuario() mide el tiempo transcurrido entre la solicitud y la respuesta. Si se alcanzan 600 segundos, un ciclo for activa la consulta para continuar con "si" o regresar al inicio con "no".

6. Captura de fecha estructurada:

Se solicitan día, mes y año por separado. Después de validar sus valores se almacena Fecha = dia, mes, anio. Esta tupla se usa al registrar mantenimiento y al anexar información.

7. Persistencia en archivos simples de texto:

Se utilizan cuatro archivos: mazda_3.txt, toyota_hilux.txt, chevrolet_suburban.txt e historial_general.txt. El sistema permite leer contenido y anexar registros mediante with open(..., "a").

8. Control de excepciones:

Las operaciones de archivo incluyen try-except para FileNotFoundError, PermissionError y OSError. También se controlan conversiones numéricas y entradas inválidas para evitar que el programa termine abruptamente

9. Depuración técnica con PDB:

Se realizaron pruebas controladas con PDB/IPDB para localizar fallas de lógica en la selección de vehículo, el tipo de mantenimiento y la validación de fechas. Las evidencias y correcciones se documentan en la sección 9.

10. Comentarios de calidad:

El código está dividido por secciones y contiene comentarios que explican variables, validaciones, ciclos,

funciones y operaciones de archivo para facilitar su mantenimiento.

Explicación del código final

Importación y variables principales:

Se importa time para medir inactividad y generar la pantalla de carga. En la versión final no se mantiene PDB activo; el módulo se utilizó únicamente durante las pruebas de depuración. Los costos de mantenimiento se almacenan en variables independientes para cada unidad y tipo de servicio.

Lista de vehículos:

La variable vehiculos contiene tres sublistas. Cada una almacena el nombre normalizado de la unidad y su tipo. Esta estructura se utiliza para mostrar las unidades disponibles al usuario.

Lista de archivos:

La lista archivos contiene los cuatro archivos de texto del proyecto. La función mostrar_archivos() recorre esta lista y presenta al usuario las opciones disponibles.

Matriz del menú:

La variable menu se construye como una lista de filas, donde cada fila contiene un número de opción y su descripción. Un ciclo for recorre la matriz para imprimirla y un while mantiene activo el menú hasta que el usuario seleccione salir.

Función de carga:

carga() muestra el texto "Cargando sistema" y utiliza un for de tres repeticiones. En cada repetición imprime un punto y realiza una pausa de un segundo.

Función de entrada y control de inactividad:

entrada_usuario(mensaje) registra el tiempo antes y después de input(). Si el tiempo alcanza diez minutos, se pregunta si el usuario desea continuar. Se aceptan "si" y "sí" para continuar, o "no" para regresar a la pantalla de inicio.

Selección del vehículo:

seleccionar_vehiculo() muestra las unidades y acepta tanto el número como el nombre. La cadena ingresada se normaliza mediante strip(), lower(), split() y join(), reduciendo errores por mayúsculas o espacios adicionales. Si la opción no corresponde con ninguna unidad, el ciclo vuelve a solicitarla.

Selección del archivo:

seleccionar_archivo() presenta los cuatro archivos y permite seleccionarlos por número o nombre. La entrada también se convierte a minúsculas y se eliminan espacios innecesarios.

Identificación y contraseña:

En el programa principal se solicita el nombre del usuario y se valida que no quede vacío. Después se solicita la contraseña. Si no coincide con el valor configurado, el sistema informa el error y regresa al inicio.

Validación de la fecha:

El día, mes y año se solicitan por separado. Primero se comprueba que sean datos numéricos y que el año tenga cuatro dígitos. Después se determina la cantidad de días válidos de acuerdo con el mes. Para febrero también se evalúa si el año es bisiesto. Una vez validada, se crea la tupla Fecha = dia, mes, anio.

Consulta de costos de mantenimiento:

La opción 1 solicita el vehículo, kilometraje actual y kilometraje del próximo servicio. Ambos kilometrajes deben ser enteros positivos. Si el actual ya alcanzó el del servicio, se solicita el tipo de mantenimiento y se muestra el costo correspondiente. En caso contrario, se calculan los kilómetros restantes.

Registro de mantenimiento:

La opción 2 solicita vehículo, kilometraje y tipo de servicio. Dependiendo de la unidad y el mantenimiento elegido se determina el archivo, costo y descripción. Posteriormente se construye un registro con fecha, usuario, vehículo, kilometraje, servicio y costo. El registro se escribe tanto en el archivo individual como en historial_general.txt.

Lectura de archivos:

La opción 3 permite seleccionar uno de los archivos. El contenido se lee con `open(..., "r")` y `read()`. Si el archivo está vacío se informa al usuario; si contiene información, se muestra en consola.

Anexado de información:

La opción 4 permite agregar texto a uno de los archivos existentes. No se permite anexar una cadena vacía. El sistema agrega automáticamente la fecha y el usuario al texto antes de escribirlo con modo "a".

Manejo de excepciones:

Las operaciones de persistencia se encuentran dentro de bloques `try-except`. `FileNotFoundError` controla archivos inexistentes, `PermissionError` controla fallos de permisos y `OSError` cubre otros errores del sistema de archivos. Las conversiones numéricas también están protegidas para evitar interrupciones por datos no válidos.

Salida del programa:

La opción 5 muestra un mensaje de cierre y cambia las variables de control `menuact` y `prog` a `False` para terminar los ciclos de forma ordenada.

Clasificación de datos y estructuras de control

Str: nombre de vehículos y contraseña

Int: Kilometrajes, costos y opciones

Tuple: Fecha

List: Vehiculos, archivos y menú

Bool: Estados de ejecución

Las principales estructuras de control utilizadas son `if`, `elif`, `else`, `while`, `for`, `try` y `except`, la combinación de estas estructuras permite validar entradas, repetir operaciones cuando existe un error, controlar el menú y responder ante excepciones sin finalizar el programa de forma inesperada.

Evidencia de depuración técnica con PDB

Para cumplir con la depuración técnica se realizaron pruebas controladas utilizando PDB/IPDB.

En cada caso se detuvo temporalmente la ejecución, se inspeccionaron variables y condiciones, se identificó la causa de la falla lógica y posteriormente se aplicó una corrección en el código final.

Nombre de vehículo escrito con mayúsculas:

Durante la prueba se ingresó Mazda 3. Al inspeccionar la variable entrada mediante el depurador se observó que conservaba la mayúscula inicial y que la comparación con "mazda 3" devolvía False. Esto explicaba por qué una entrada entendible para el usuario podía no coincidir con la condición del programa.

```
MENU PRINCIPAL
1 - Consultar costos del mantenimiento
2 - Registrar el mantenimiento
3 - Leer un archivo
4 - Anexar nueva informacion
5 - Salir del programa

UNIDADES DISPONIBLES
1 - Mazda 3 - Auto utilitario
2 - Toyota Hilux - Vehiculo utilitario de carga
3 - Chevrolet Suburban - Auto ejecutivo

-----
CONSULTAR COSTO DE MANTENIMIENTO

VEHICULOS REGISTRADOS
1 - Mazda 3 - Auto utilitario
2 - Toyota Hilux - Vehiculo utilitario de carga
3 - Chevrolet Suburban - Auto ejecutivo
> /var/folders/tk/x9qh13g11t501kzm5460xpqh0000gn/T/ipykernel_15749/2083613216.py(163)seleccionar_vehiculo()
161
162         # Mazda 3
-> 163         if entrada == "1":
164             return "mazda 3"
165

'Mazda 3'
False
```

Por lo que se corrigió de la siguiente manera:

```
entrada = " ".join(
    entrada.strip().lower().split()
)
```

Python

La corrección normaliza la entrada antes de compararla. strip() elimina espacios al inicio y al final, lower() convierte el texto a minúsculas y

split() junto con join() elimina espacios adicionales entre palabras. Con esto, entradas como Mazda 3, MAZDA 3 o " Mazda 3 " se interpretan de la misma manera.

Tipo de mantenimiento inválido:

Durante la prueba se ingresó el valor 3 como tipo de mantenimiento. Mediante el depurador se verificó que la variable tipo contenía '3' y que las comparaciones con "1" y "2" devolvían False. Esto mostró que era necesario controlar de forma explícita las entradas que no correspondieran a las opciones permitidas.

```
-----
CONSULTAR COSTO DE MANTENIMIENTO

VEHICULOS REGISTRADOS
1 - Mazda 3 - Auto utilitario
2 - Toyota Hilux - Vehiculo utilitario de carga
3 - Chevrolet Suburban - Auto ejecutivo

Vehiculo seleccionado: Mazda 3
Tipo de vehiculo: Auto utilitario

La unidad requiere mantenimiento
1 - Mantenimiento menor
2 - Mantenimiento mayor
> /var/folders/tk/x9qh13q11t501kzm5460xpqh0000gn/T/ipykernel_15903/2254574.py(631)<module>()
629
630         if (
--> 631             tipo == "1"
632             or tipo == "mantenimiento menor"
633             or tipo == "menor"

'3'
False
False
```

Por lo que se corrigió de la siguiente manera:

```
else:
    print(
        "Tipo de mantenimiento no valido. "
        "Selecciona 1 o 2."
    )
    Continue
```

Python

La falla se corrigió agregando una rama else que muestra un mensaje de error cuando el usuario ingresa una opción no válida. Con continue, el programa evita seguir con una selección incorrecta y regresa al flujo del menú sin asignar un costo o servicio que no corresponda.

Fecha no posible:

Durante la prueba se ingresó la fecha 31/02/2026. Mediante el depurador se verificó que día tenía el valor 31, mes el valor 2 y año el valor 2026. Las validaciones básicas día <= 31 y mes <= 12 devolvieron True, por lo que los valores eran aceptados individualmente aunque la fecha completa no existe.

```
=====
SISTEMA PARA EL MANTENIMIENTO DE FLOTILLAS CEMEX
=====
Bienvenido al sistema edu
Acceso permitido
> /var/folders/tk/x9qh13q11t501kzm5460xpqh0000gn/T/ipykernel_16093/3824939086.py(406)<module>()
404
405      # La fecha se almacena en una tupla.
--> 406      Fecha = dia, mes, anio
407
408

31
2
2026
True
True
```

Por lo que se corrigió de la siguiente manera:

```
dias_mes = 31

if mes == 4 or mes == 6 or mes == 9 or mes == 11:
    dias_mes = 30

elif mes == 2:
    if anio % 400 == 0 or (anio % 4 == 0 and anio % 100 != 0):
        dias_mes = 29
    else:
        dias_mes = 28

if dia < 1 or dia > dias_mes:
    print("El día ingresado no es válido para ese mes y año")
    continue
```

Con esta modificación, el programa valida la cantidad real de días de cada mes y también considera los años bisiestos. Por lo tanto, fechas imposibles como 31/02/2026 ya no son aceptadas.

Conclusiones

La fase final permitió transformar el prototipo inicial en un sistema de consola más completo y resistente a errores de captura. Además de

determinar si una unidad requiere mantenimiento y consultar costos, el programa conserva información en archivos de texto, integra automáticamente la fecha de operación y controla diferentes errores mediante validaciones y excepciones.

La depuración con PDB permitió localizar fallas de lógica que no necesariamente provocaban un error de Python, pero sí podían generar resultados incorrectos para el usuario. Las correcciones realizadas en la normalización de nombres, validación del tipo de mantenimiento y captura de fechas mejoraron la confiabilidad del programa.

El proyecto final mantiene una estructura sencilla y comprensible, utilizando los recursos revisados durante el curso: variables, cadenas, listas, tuplas, matrices, condicionales, ciclos, funciones, archivos externos, manejo de excepciones y depuración.